API_REFERENCE

ClickUp API Reference — clickup_sync design input

Field	Value
Revision	1
Last modified	2026-06-09T00:00:00Z
Phase	0 — research only (no code, no writes to ClickUp)
Anti-bluff	§11.4.6 — every API fact carries an official-doc URL or is marked `UNCONFIRMED:`

> Probe method: read-only `curl -H "Authorization: $CLICKUP_API_KEY"` GET calls against `api.clickup.com/api/v2` + WebFetch of `developer.clickup.com`. The API key is `pk_…` (45 chars), never echoed/logged/committed per §11.4.10.

(a) v2 vs v3/beta status, base URLs, auth

- **v2 base URL:** `https://api.clickup.com/api/v2` — the stable, fully-documented surface. SOURCE: https://developer.clickup.com/docs/authentication, https://developer.clickup.com/docs/Getting%20Started
- **v3:** exists for only a handful of endpoints; "the rest of the API is still being updated to v3". Treat v2 as the integration target; v3 is `UNCONFIRMED:` for the resources we need (tasks/statuses/webhooks). SOURCE: https://developer.clickup.com/docs/Getting%20Started
- **Auth — personal token:** token begins `pk_`, sent as a **raw** `Authorization` header value, **NO `Bearer` prefix**: `Authorization: pk_xxx`. Personal tokens never expire. SOURCE: https://developer.clickup.com/docs/authentication
  - Empirically confirmed: `GET /api/v2/folder/901814301358` with `Authorization: pk_…` → **HTTP 200** (this probe run, 2026-06-09).

- **OAuth alternative** (`Get Access Token`) exists for multi-workspace apps; not needed for a single-team internal sync. SOURCE: https://developer.clickup.com/reference/getaccesstoken

(b) Rate limits, 429, Retry-After

SOURCE: https://developer.clickup.com/docs/rate-limits

Plan	Requests / minute / token
Free Forever, Unlimited, Business	**100**
Business Plus	1,000
Enterprise	10,000

- On exceed → **HTTP 429**.
- Response headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` (reset = **Unix timestamp**, seconds).
- **`Retry-After: UNCONFIRMED:`** — the rate-limits doc page does NOT mention a `Retry-After` header. Design MUST NOT assume one; back off using `X-RateLimit-Reset` (sleep until reset epoch) as the authoritative signal, with exponential fallback if the header is absent. (tracked-task: confirm against a live 429 before relying on it.)
- **Design note:** our workspace is team `90182716341` = "FinTech Labs" — plan tier `UNCONFIRMED:` (not exposed by `GET /team`); assume the **100 req/min** floor and engineer to it.

(c) Hierarchy model + Board↔List mapping

`Workspace (a.k.a. Team)` → `Space` → `Folder` → `List` → `Task` → `Subtask`. A Folder may hold many Lists; Lists may also be folderless (`GET /api/v2/space/{space_id}/list`). SOURCE: https://developer.clickup.com/docs/Getting%20Started

- **Board view ↔ List:** a ClickUp **Board view is a presentation of a List's tasks**, one card-column per task **status**. The API has no "board" resource for tasks — you read/write the underlying **List** and its **status set**. The columns ARE the List's statuses (see §e). The board-view URL `…/v/b/5-<folderId>-2` encodes the folder/view, not a separate task container.
- Endpoints used to walk the tree:
  - `GET /api/v2/team` — authorized workspaces + members
  - `GET /api/v2/team/{team_id}/space` — spaces (NOTE: `GET /space/{id}` directly also works)
  - `GET /api/v2/folder/{folder_id}` — folder + its lists (embedded)

    - `GET /api/v2/folder/{folder_id}/list` — lists in a folder
    - `GET /api/v2/space/{space_id}/list` — folderless lists
    - `GET /api/v2/list/{list_id}` — list + its statuses SOURCE: https://developer.clickup.com/llms.txt

(d) Task CRUD

SOURCE: https://developer.clickup.com/llms.txt + https://developer.clickup.com/docs/tasks

Operation	Method + path
Get Tasks (one List, paginated)	`GET /api/v2/list/{list_id}/task`
Get Task (single)	`GET /api/v2/task/{task_id}`
Create Task	`POST /api/v2/list/{list_id}/task`
Update Task	`PUT /api/v2/task/{task_id}`
Delete Task	`DELETE /api/v2/task/{task_id}`
Get Filtered Team Tasks (cross-list, incremental)	`GET /api/v2/team/{team_id}/task`

- **Important:** `Update Task` (`PUT /task/{id}`) does **NOT** update Custom Field values — custom fields use a separate endpoint (see §l). SOURCE: https://developer.clickup.com/docs/tasks

(e) Custom statuses per List + read/create + ORDER

- Statuses live on the **Space** and may be **overridden per List** (`override_statuses` boolean on the List). `GET /api/v2/list/{list_id}` returns `statuses[]` each with `status` (name), `orderindex` (the **board column order**), `type` (`open` | `custom` | `closed` | `done`), `color`. SOURCE: https://developer.clickup.com/llms.txt; field shape empirically confirmed this run (see BOARD_SNAPSHOT.md).
- **Read order:** `orderindex` ascending = left-to-right board columns.
- **Create / change the status set:** there is **NO dedicated public v2 "create status" endpoint**. Statuses are managed by updating the **Space** (`PUT /api/v2/space/{space_id}` with a `statuses` array) or the **List** when `override_statuses=true`. `UNCONFIRMED`: whether `PUT /space` reliably mutates the status set via public API — ClickUp's status management is partly UI-

only. **Design decision: the ClickUp board is the source-of-truth for the status SET; clickup_sync MIRRORS ClickUp statuses into our SQLite vocabulary, and does NOT attempt to author ClickUp statuses programmatically** unless a confirmed endpoint is found (tracked-task).
- Today's live set (inherited from Space, `override_statuses=False`): `to do` (open, order 0) → `in progress` (custom, order 1) → `complete` (closed, order 2).

(f) Task types / custom item types

- `GET /api/v2/team/{team_id}/customitem` (a.k.a. `custom_item`) returns the workspace's custom task types. A task carries `custom_item_id`; **`0` = default Task type**. Other IDs = custom types. SOURCE: https://developer.clickup.com/llms.txt, https://developer.clickup.com/reference/getcustomitems
- **Enterprise/paid feature.** Our Space reports `features.custom_items.enabled = False` (this run) → only the default Task type is available. **Design: do NOT depend on custom item types.**

(g) Tags vs Labels vs Custom Fields — the multi-value "version tags" mechanism

Three candidate mechanisms for "a task can carry several version tags":

1. **Tags** (Space-level, multi-value, native): `GET /api/v2/space/{space_id}/tag` lists tags; `POST /api/v2/task/{task_id}/tag/{tag_name}` adds a tag; `DELETE /api/v2/task/{task_id}/tag/{tag_name}` removes. A task's `tags[]` is inherently multi-value. Tags must exist at Space scope. SOURCE: https://developer.clickup.com/llms.txt. Space tags feature `enabled=True` (this run); current tag set is **empty**.
2. **Labels custom field** (`type: "labels"`): a multi-select custom field — multiple option ids per task. Read/write via the custom-field endpoints (§l).
3. **Drop-down custom field** (`type: "drop_down"`): single-value (NOT suitable for multiple version tags).

Recommendation for "version tags a task may have several of": use **native Tags** (mechanism 1) — they are first-class, multi-value, space-scoped, surface on the board card, and have dedicated add/remove endpoints that don't require the

`set custom field value` dance. A `labels` custom field is the fallback if we need per-list scoping or a constrained option set. (No custom fields or tags exist on the target yet — clean slate.)

(h) `date_updated` / last-edited + who made the last edit (conflict resolution)

- `GET /api/v2/task/{task_id}` returns `date_created`, `date_updated`, `date_closed`, `date_done` — each a **Unix epoch milliseconds STRING**. `UNCONFIRMED:` exact ms-string formatting not re-derived from a live task (target list has 0 tasks); strongly documented as ms strings across ClickUp refs — treat as ms string, parse defensively. SOURCE: https://developer.clickup.com/reference/gettask (schema not rendered by WebFetch — flagged), corroborated by https://developer.clickup.com/llms.txt task model.
- **WHO made the last edit:** the **Task object does NOT expose a "last editor" field** — it exposes `creator` (id/username/email) only. The identity of the last editor is available **only via webhook `history_items[].user`** (each history item names the acting `user`) or via task `history`/activity. SOURCE: https://developer.clickup.com/docs/webhooks (payload `history_items` carries `user`, `before`, `after`). **Design consequence:** for conflict resolution, last-editor attribution comes from the **webhook history_item.user**, NOT from the task GET. The task GET gives only `date_updated` (the "newer side" timestamp). This is load-bearing for RISK_ANALYSIS.md.

(i) Members / assignees / creators (user id → name)

- `GET /api/v2/team` returns each workspace's `members[].user` with `id`, `username`, `email`, `role` (1=owner, 2=admin, 3=member, 4=guest — `UNCONFIRMED:` exact numeric→role map; observed roles 1 and 3 this run). SOURCE: https://developer.clickup.com/llms.txt. Live members captured in RESOLVED_IDS.md.
- Task `assignees[]`, `watchers[]`, `creator` are user objects (`id,username,email,profilePicture`). Resolve id→name from the cached `GET /team` member roster.
- `multiple_assignees=True` on our Space → tasks may have several assignees.

(j) Webhooks

SOURCE: https://developer.clickup.com/docs/webhooks, https://developer.clickup.com/docs/webhooksignature, https://developer.clickup.com/llms.txt

- **Register:** `POST /api/v2/team/{team_id}/webhook` with `{ endpoint, events[], (optional) space_id/folder_id/list_id/task_id scope }`. Response returns `id` + **`secret`** (the HMAC key — store per §11.4.10).
- **Manage:** `GET /api/v2/team/{team_id}/webhook`, `PUT /api/v2/webhook/{webhook_id}`, `DELETE /api/v2/webhook/{webhook_id}`.
- **Event set (task-relevant):** `taskCreated`, `taskUpdated`, `taskDeleted`, `taskStatusUpdated`, `taskPriorityUpdated`, `taskAssigneeUpdated`, `taskDueDateUpdated`, `taskTagUpdated`, `taskMoved`, `taskCommentPosted`, `taskCommentUpdated`, `taskTimeEstimateUpdated`, `taskTimeTrackedUpdated`. Plus list/folder/space/goal events. (Use `*` / all-events or an explicit subset.)
- **Payload shape:** top-level `event`, `task_id` (or `list_id` etc.), `webhook_id`, and `history_items[]` each with `id`, `type`, `date`, `field`, `source`, `user` (the actor), `before`, `after`. `taskDeleted` carries **only `task_id`** (no before/after detail) — SOURCE: https://feedback.clickup.com/feature-requests/p/get-the-more-details-on-taskdeleted-webhook-event.
- **Signature:** header **`X-Signature`** = `HMAC-SHA256(webhook.secret, raw_request_body)` digested as **hexadecimal**. Verify against the **raw** body bytes (stringify without added whitespace). Node: `crypto.createHmac('sha256', secret).update(rawBody).digest('hex')`. SOURCE: https://developer.clickup.com/docs/webhooksignature.
- **Delivery guarantees / retry:** `UNCONFIRMED:` — the webhooks doc defers retry/health detail to `…/docs/webhookhealth` (not fetched this run). ClickUp tracks a per-webhook health/fail status and disables after repeated failures (general knowledge, tracked-task to confirm exact retry count + backoff). **Design MUST treat webhooks as at-least-once and possibly-missed** → mandates the reconciling full-sweep backstop (RISK_ANALYSIS.md).

(k) Delete + trash retrievability ("Deleted" status mapping)

- `DELETE /api/v2/task/{task_id}` moves the task to **Trash**; trashed items are retained **30 days** then permanently purged. SOURCE: https://help.clickup.com/hc/en-us/articles/6311358652951-Delete-a-task, https://clickup.com/api/clickupreference/operation/DeleteTask/
- **There is NO public API to read the Trash / list deleted tasks** — it is an open ClickUp feature request. SOURCE: ht-

tps://feedback.clickup.com/public-api/p/have-trash-items-available-through-api, https://feedback.clickup.com/feature-requests/p/retrieve-deleted-tasks-trash-bin-3uf. **Design consequence:** a deletion is observed ONLY via the `taskDeleted` webhook (task_id only) OR by a task disappearing from a full-sweep GET. clickup_sync CANNOT poll the trash; it MUST record the deletion event itself. Map ClickUp delete → our `Deleted/Obsolete` state by capturing the `taskDeleted` webhook (and a sweep-diff backstop), never by reading trash.

(l) Custom fields — create + set value

- **Read defs:** `GET /api/v2/list/{list_id}/field` → `fields[]` (`id`, `name`, `type`, `type_config.options[]`). SOURCE: https://developer.clickup.com/llms.txt. Target list: **0 custom fields** (this run).
- **Set value:** `POST` or `PUT /api/v2/task/{task_id}/field/{field_id}` with `{ value }` (the llms index shows `PUT`; ClickUp commonly accepts both — `UNCONFIRMED:` exact verb, use `POST` per the canonical "Set Custom Field Value" ref and accept 200).
- **Remove value:** `DELETE /api/v2/task/{task_id}/field/{field_id}`.
- **Create a custom field DEFINITION:** there is **NO documented public v2 endpoint to create a custom field** (creation is UI-only). `UNCONFIRMED:` — tracked-task. **Design: clickup_sync MIRRORS existing ClickUp custom fields; if our sync-marker metadata field (see RISK_ANALYSIS.md) cannot be created via API, the operator creates it once in the UI and we reference it by id.**

(m) Pagination + bulk fetch

- List tasks + filtered team tasks paginate via **`page` (starts at 0), 100 tasks/page**; iterate until a short/empty page. SOURCE: https://developer.clickup.com/reference/getfilteredteamtasks.
- **Incremental sync key:** `GET /api/v2/team/{team_id}/task?list_ids[]=<list>&date_updated_gt=<ms>&order_by=updated&subtasks=true&include_closed=true&page=N` — poll everything changed since our last-seen `date_updated`. This is the cron full-sweep / catch-up backstop for missed webhooks. SOURCE: https://developer.clickup.com/reference/getfilteredteamtasks.

Open / UNCONFIRMED items (tracked for Phase 1)

1. `Retry-After` header on 429 (use `X-RateLimit-Reset` regardless).
2. Exact webhook retry/health semantics (…`/docs/webhookhealth`).
3. Whether public API can CREATE statuses / custom-field defs / tags-as-options (assume NOT; operator-seeds in UI).

4. Exact `date_updated` ms-string formatting (re-derive from a live task in Phase 1).
5. `PUT` vs `POST` verb for set-custom-field-value.
6. Numeric `role` → name map (owner/admin/member/guest).